CS5293 Final Exam Review / 期末复习资料

Topics on Information Security --- Prof. Cong Wang, CityU CS
Coverage: Weeks 1--11 (Cryptography, PKI/TLS, Authentication, Memory Corruption, DNS, Firewalls/VPN, Web Security, CSRF/SSRF/SQLi, AI Security)

1. Knowledge Framework / 知识框架

Week	Topic / 主题	Core Focus / 核心要点
1	Cryptography Foundations / 密码学基础	Symmetric/Asymmetric crypto, AES modes, hash/MAC
2	PKI, TLS & Secure Channels / 公钥基础设施与安全通道	Certificate chain, TLS handshake, attacks
3	Authentication & Access Control / 认证与访问控制	Password security, Set-UID, RBAC/MAC/DAC
4	Memory Corruption: Basics / 内存破坏基础	Stack overflow, shellcode, ASLR/Canary/NX
5	Advanced Memory Corruption / 高级内存破坏	Ret2libc, ROP, Format String, PLT/GOT
6	OpenClaw & AI Agents / AI代理安全 (supplementary)	Agent loop, attack surfaces, local deployment
7	DNS Security / DNS安全	Cache poisoning, Kaminsky, DNSSEC, DoH/DoT
8	Firewalls & VPN / 防火墙与VPN	iptables, stateful filtering, TUN/TAP, IPsec/OpenVPN
9	Web Security & XSS / Web安全与XSS	SOP, Reflected/Stored/DOM XSS, CSP
10	CSRF, SSRF & SQL Injection / CSRF/SSRF/SQL注入	CSRF tokens, SSRF metadata, prepared statements
11	Security of AI / AI安全	Jailbreaks, prompt injection, defense in depth
12	AI for Cybersecurity / AI赋能网络安全	AI offense/defense, agent reliability, red/blue/purple teaming

2. Core Concepts / 核心概念梳理

2.1 Cryptography / 密码学 (Week 1)

Symmetric Encryption (对称加密)
- Definition: Same key for encryption and decryption / 加密和解密使用同一密钥
- Key Points: Fast; key distribution problem / 速度快；密钥分发问题
- AES: Block cipher, 128/192/256-bit keys / 分组密码，128/192/256位密钥

AES Modes of Operation (AES工作模式)

Mode	Parallel Encrypt	Parallel Decrypt	Random Access	Error Propagation	Security
ECB	Yes	Yes	Yes	Single block	Insecure — identical plaintext → identical ciphertext
CBC	No	Yes	No	Two blocks	Secure with random IV
CTR	Yes	Yes	Yes	None	Secure if nonce never reused
GCM	Yes	Yes	Yes	None	Authenticated encryption (AEAD)

ECB Insecurity (ECB模式不安全性)
- Identical plaintext blocks produce identical ciphertext blocks / 相同明文块产生相同密文块
- Reveals patterns in data (e.g., images still recognizable) / 泄露数据模式
- CBC solves this by XORing each block with previous ciphertext before encryption / CBC通过与前一个密文块异或来解决

Hash Functions (哈希函数)
- Definition: One-way function: easy to compute, infeasible to reverse / 单向函数：易计算，不可逆
- Properties: Pre-image resistance, second pre-image resistance, collision resistance / 原像抗性、第二原像抗性、碰撞抗性
- Examples: SHA-256, SHA-3, MD5 (broken) / SHA-256、SHA-3、MD5 (已破解)

MAC & HMAC (消息认证码)
- MAC: Symmetric key + hash → verify integrity AND authenticity / 对称密钥+哈希→验证完整性和真实性
- HMAC: Hash-based MAC: HMAC(K, m) = H((K⊕opad) ‖ H((K⊕ipad) ‖ m))
- Key difference: Hash verifies integrity only; MAC also authenticates sender / 哈希仅验证完整性；MAC还认证发送者

Diffie-Hellman Key Exchange (Diffie-Hellman密钥交换)
- Two parties agree on shared secret without sending it / 双方不发送共享密钥即可协商共享秘密
- Vulnerable to Man-in-the-Middle (MITM) attack / 易受中间人攻击
- Authenticated DH (with certificates) prevents MITM / 认证DH（带证书）防止MITM

2.2 PKI & TLS / 公钥基础设施与TLS (Week 2)

Public-Key Cryptography (公钥密码学)
- Asymmetric: Public key encrypts, private key decrypts / 非对称：公钥加密，私钥解密
- RSA: Trapdoor function, 2048-4096 bit keys / 陷阱门函数，2048-4096位密钥
- Hybrid approach: Asymmetric for key exchange, symmetric for bulk data / 混合方式：非对称用于密钥交换，对称用于批量数据

Certificate & PKI (数字证书与公钥基础设施)
- Certificate: Binds public key to identity, signed by CA / 将公钥绑定到身份，由CA签名
- Certificate Chain: Root CA → Intermediate CA → End-entity cert / 证书链：根CA→中间CA→终端证书
- Trust Store: OS/browser ships with trusted Root CA public keys / 信任库：操作系统/浏览器内置可信根CA公钥

TLS Handshake (TLS握手)
1. ClientHello: Supported cipher suites, random nonce / 支持的密码套件，随机数
2. ServerHello: Chosen cipher suite, certificate / 选定密码套件，证书
3. Key Exchange: DH or RSA-based pre-master secret / DH或RSA方式的预主密钥
4. Derive session keys: From pre-master secret + both randoms / 从预主密钥+双方随机数派生会话密钥
5. Finished: Encrypted handshake messages / 加密的握手消息

TLS Attacks (TLS攻击)
- SSL Stripping: Downgrade HTTPS→HTTP (Moxie Marlinspike) / 降级HTTPS为HTTP
- HSTS: HTTP Strict Transport Security; tells browser to always use HTTPS / HTTP严格传输安全；告诉浏览器始终使用HTTPS
- HSTS limitation: First visit still vulnerable (no HSTS header received yet) / 首次访问仍有漏洞
- BEAST/CRIME/Poodle: Protocol-level attacks on older TLS versions / 针对旧版TLS的协议级攻击

SNI & ECH (服务器名称指示与加密SNI)
- SNI: Client sends hostname in plaintext in ClientHello for virtual hosting / 客户端在ClientHello中明文发送主机名
- ECH: Encrypts SNI to prevent observers from seeing which site / 加密SNI防止观察者看到访问的网站

2.3 Authentication & Access Control / 认证与访问控制 (Week 3)

Three Authentication Factors (三种认证因素)

Factor	Type	Examples
Something you know	Knowledge / 知识	Password, PIN
Something you have	Possession / 持有	Phone, hardware token
Something you are	Biometric / 生物	Fingerprint, face

Biometric Metrics (生物识别指标)
- FAR (False Acceptance Rate): Unauthorized accepted / 未授权被接受
- FRR (False Rejection Rate): Authorized rejected / 授权被拒绝
- CER (Crossover Error Rate): FAR = FRR; lower = better system / FAR=FRR；越低系统越好

Password Security (密码安全)
- Storage: Never store plaintext; use hash + salt / 永不存储明文；使用哈希+盐
- Rainbow table: Precomputed hash lookup table / 预计算哈希查找表
- Salt: Random value per user; defeats rainbow tables / 每用户随机值；击败彩虹表
- KDF (Key Derivation Function): Slow hash (bcrypt, scrypt, Argon2) for brute-force resistance / 慢哈希用于抗暴力破解

Access Control Models (访问控制模型)

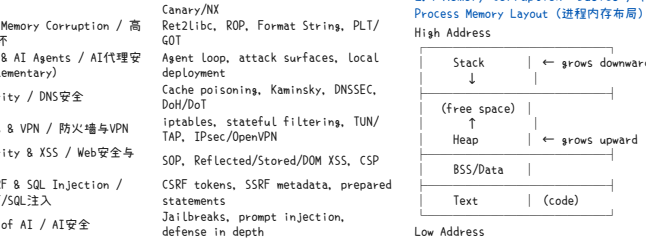
Model	Principle / 原则	Example
DAC	Owner controls access / 所有者控制访问	Unix file permissions
MAC	System-enforced labels / 系统强制标签	Military classification
RBAC	Role-based permissions / 基于角色的权限	Corporate IT systems
Bell-LaPadula	No read up, no write down (confidentiality) / 不上读不下写	

Set-UID Mechanism (Set-UID机制)
- RUID (Real UID): Who launched the process / 谁启动了进程
- EUID (Effective UID): Whose privileges are used / 使用谁的权限
- Set-UID bit: Program runs with owner's EUID / 程序以所有者的EUID运行
- chmod 4755: 4 = Set-UID bit + 755 (rwxr-xr-x)

Set-UID Attack Surfaces (Set-UID攻击面)
- Capability Leaking: Open file descriptors persist after privilege drop / 特权降级后文件描述符仍开放
- Environment Variables: PATH manipulation, LD_PRELOAD / PATH操纵, LD_PRELOAD
- Command Injection: system() with user input / 使用用户输入的system()

2.4 Memory Corruption: Basics / 内存破坏基础 (Week 4)

Process Memory Layout (进程内存布局)



Key Registers (关键寄存器)

Register	Purpose / 用途
EIP/RIP	Instruction pointer — controls execution flow / 指令指针—控制执行流
ESP/RSP	Stack pointer — top of stack / 栈指针—栈顶
EBP/RBP	Base pointer — frame anchor / 基址指针—栈帧锚点

Stack Operation (栈操作)
- CALL: Push return address, jump to function / 压入返回地址，跳转到函数
- RET: Pop return address to EIP / 弹出返回地址到EIP
- Stack grows DOWNWARD (toward lower addresses) / 栈向下增长

Buffer Overflow (缓冲区溢出)
- Writing past buffer boundary → overwrite saved EBP, return address, local variables / 越界写入→覆盖保存的EBP、返回地址、局部变量
- No bounds checking in C: gets(), strcpy(), sprintf() / C语言无边界检查

Shellcode (Shell代码)
- Machine code to spawn a shell (/bin/sh) / 生成shell的机器码
- Must be NULL-free (0x00 terminates string operations) / 必须无NULL字节

Defenses (防御机制)

Defense	Mechanism / 机制	Bypass / 绕过
ASLR	Randomizes memory layout / 随机化内存布局	Brute force (32-bit), info leak
Stack Canary	Secret value before return addr / 返回地址前的秘密值	Info leak, format string
NX/DEP	Stack not executable / 栈不可执行	Ret2libc, ROP
Shadow Stack	Hardware backup of return addresses / 硬件备份返回地址	—
PIE	Position-independent executable / 位置无关可执行文件	Info leak

2.5 Advanced Memory Corruption / 高级内存破坏 (Week 5)

Dynamic Linking: PLT/GOT (动态链接：PLT/GOT)
- PLT (Procedure Linkage Table): Stub code for lazy resolution / 延迟解析的存根代码
- GOT (Global Offset Table): Contains resolved function addresses / 包含已解析的函数地址
- Lazy binding: First call → resolver fills GOT; subsequent calls use GOT directly / 懒绑定：首次调用→解析器填充GOT；后续调用直接使用GOT

Ret2Libc (返回Libc攻击)
- Instead of injecting shellcode, return to existing libc functions / 不注入shellcode，而是返回到已有libc函数
- Chain: system("/bin/sh") using stack frames / 使用栈帧链接system("/bin/sh")
- Bypasses NX: no code on stack is executed / 绕过NX：栈上无代码执行

Return-Oriented Programming — ROP (返回导向编程)
- Gadget: Short instruction sequence ending in RET / 以RET结尾的短指令序列
- Chain gadgets via stack: each RET jumps to next gadget / 通过栈链接gadget：每个RET跳到下一个
- Turing-complete: Can perform arbitrary computation / 图灵完备：可执行任意计算
- x64 calling convention: Arguments in registers (rdi, rsi, rdx, rcx) / x64调用约定：参数在寄存器中

Format String Vulnerability (格式化字符串漏洞)
- printf(user_input): User controls format string / 用户控制格式字符串
- Reading memory: %x leaks stack values, %s reads pointer target / %x泄露栈值，%s读取指针目标
- Writing memory: %n writes count of bytes printed so far / %n写入已打印的字节数
- Direct parameter access: %7xaccesses7thparameterdirectly/x直接访问第7个参数

Modern Exploit Workflow (现代利用流程)
1. Leak: Use format string or GOT read to leak libc address / 使用格式化字符串或GOT读取泄露libc地址
2. Calculate: Compute system(), "/bin/sh" offsets from base / 从基地址计算system()、"/bin/sh"偏移
3. Pwn: Overwrite GOT or return address with computed target / 用计算目标覆盖GOT或返回地址

RELRO (重定位只读)
- Partial RELRO: GOT.PLT still writable / GOT.PLT仍可写
- Full RELRO: Entire GOT marked read-only at startup; blocks GOT overwrite / 启动时整个GOT标记为只读；阻止GOT覆写

2.6 DNS Security / DNS安全 (Week 7)

DNS Infrastructure (DNS基础设施)
- Hierarchy: Root (.) → TLD (.com) → Second-level (example) → Subdomain (www) / 层级结构
- Root servers: 13 clusters (A--M), hundreds via anycast / 13个集群，通过任播扩展
- DNS Zones: Delegation of authority / 权限委托

DNS Record Types (DNS记录类型)

Type	Purpose / 用途
A	Name → IPv4 address / 名称→IPv4地址
AAAA	Name → IPv6 address / 名称→IPv6地址
NS	Delegated name server / 委派名称服务器
MX	Mail server / 邮件服务器
CNAME	Alias / 别名
SOA	Zone authority info / 区域权威信息

DNS Query Process (DNS查询过程)
1. Client → Local resolver (stub) → Recursive resolver → Root → TLD → Authoritative
2. Caching: Each level caches responses for TTL duration / 每级缓存响应直到TTL过期
3. Iterative vs Recursive: Resolver does iterative lookup on behalf of client / 解析器代表客户端做迭代查询

DNS Cache Poisoning (DNS缓存投毒)
- Local attack: Attacker on same LAN, forge DNS response / 同一局域网，伪造DNS响应
- Remote attack (Kaminsky): Query random subdomains → never cached → unlimited attempts / 查询随机子域名→永不缓存→无限尝试
- Key insight of Kaminsky: Random subdomains bypass caching, giving unlimited fresh attempts / 随机子域名绕过缓存，获得无限新鲜尝试
- Required fields to spoof: Source port + Transaction ID (16-bit each) / 源端口+事务ID

Modern DNS Threats (现代DNS威胁)

Threat	Description / 描述
DGA	Domain Generation Algorithm — malware uses algorithmic domains / 域名生成算法
Fast Flux	Rapidly changing IP addresses behind a domain / 域名后快速更换IP
DNS Tunneling	Encode data in DNS queries/responses for exfiltration / 在DNS查询/响应中编码数据以进行数据外泄
DNS Rebinding	Manipulate DNS to bypass same-origin policy / 操纵DNS绕过同源策略

DNSSEC (DNS安全扩展)
- Signs DNS records with digital signatures / 用数字签名签署DNS记录
- Chain of trust: Root KSK → ZSK → Zone signatures / 信任链：根KSK→ZSK→区域签名
- DS record: Links parent zone to child zone's KSK / DS记录将父区域链接到子区域的KSK
- Does NOT encrypt DNS data; only provides authenticity and integrity / 不加密DNS数据；仅提供真实性和完整性

Encrypted DNS (加密DNS)
- DoH (DNS over HTTPS): Port 443; blends with web traffic / 端口443；与Web流量混合
- DoT (DNS over TLS): Port 853; dedicated port / 端口853；专用端口
- Detection challenge: DoH looks like normal HTTPS / DoH看起来像正常HTTPS

2.7 Firewalls & VPN / 防火墙与VPN (Week 8)

Firewall Types (防火墙类型)

Type	Works at	Inspects / 检查
Packet filtering	Network layer (L3)	IP, port, protocol headers
Stateful firewall	L3-L4 + state	Connection tracking (conntrack)
Application proxy	L7	Full application data

iptables Architecture (iptables架构)
- Tables: filter (default), nat, mangle / filter (默认)、nat、mangle
- Chains: INPUT, OUTPUT, FORWARD / INPUT、OUTPUT、FORWARD
- Rules: Match criteria → target (ACCEPT, DROP, REJECT) / 匹配条件→目标
- Connection tracking: ESTABLISHED, RELATED states allow return traffic / ESTABLISHED、RELATED状态允许返回流量

Stateful vs Stateless Filtering (有状态vs无状态过滤)
- Stateless: Only checks individual packets; easily spoofed / 仅检查单个包；易被伪造
- Stateful: Checks connection state; blocks unsolicited inbound / 检查连接状态；阻止未经请求的入站

VPN Concepts (VPN概念)
- TUN interface: Layer 3 (IP packets) --- no Ethernet overhead / 三层（IP包）---无以太网开销
- TAP interface: Layer 2 (Ethernet frames) --- bridging, same broadcast domain / 二层（以太网帧）---桥接，同广播域
- TUN preferred for site-to-site routing; TAP when L2 connectivity needed / TUN用于站点间路由；TAP在需要L2连接性时使用

VPN Packet Flow (VPN包流程)
1. Application sends packet to private network (e.g., 10.0.8.5) / 应用发送包到私有网络
2. Kernel routing table directs traffic to tun0 / 内核路由表将流量导向tun0
3. VPN app reads raw packet from tun0, encrypts it / VPN应用从tun0读取原始包，加密
4. Wraps in new IP/UDP packet addressed to VPN server's public IP / 封装在新的IP/UDP包中
5. Sends out through eth0 / 通过eth0发出

IPsec vs OpenVPN

Feature	IPsec	OpenVPN
Standard	IETF standard	Proprietary protocol
NAT traversal	ESP in UDP (port 4500)	UDP port 1194
Compatibility	Kernel-level, varies	User-space, cross-platform
Firewall traversal	Can be blocked	Looks like regular UDP

2.8 Web Security & XSS / Web安全与XSS (Week 9)

Same-Origin Policy — SOP (同源策略)
- Origin = scheme + host + port / 源 = 协议 + 主机 + 端口
- SOP blocks: DOM access, AJAX reads across origins / SOP阻止跨源的DOM访问和AJAX读取
- SOP allows: Cross-origin form submission, script/image loading / SOP允许跨源表单提交、脚本/图片加载

XSS Types (XSS类型)

Type	Source	Persistence / 持久性
Reflected	URL parameter, reflected in response / URL参数，反映在响应中	One-time / 一次性
Stored	Saved in database (e.g., profile field) / 保存在数据库中	Persistent / 持久
DOM-based	Client-side JS writes user input to DOM without sanitization / 客户端JS将用户输入写入DOM	Client-side only / 仅客户端

XSS Impact (XSS影响)
- Steal cookies (if not HttpOnly) / 窃取Cookie
- Bypass CSRF tokens (XSS can read page content) / 绕过CSRF令牌
- Keylogging, phishing, worm propagation / 键盘记录、钓鱼、蠕虫传播
- Key insight: XSS injects code into trusted origin → full access to victim's session / XSS在可信源中注入代码→完全访问受害者会话

XSS Defenses (XSS防御)

Defense	Mechanism / 机制
Output Encoding	Context-aware escaping (HTML, JS, URL, CSS) / 上下文感知转义
CSP	Content Security Policy — restrict script sources / 内容安全策略—限制脚本来源
HttpOnly	Cookie flag — JS cannot access cookie / Cookie标志—JS无法访问
Framework auto-escaping	React, Django escape by default / React、Django默认转义

CSP Directives (CSP指令)
- default-src 'self': Only load from same origin / 仅从同源加载
- script-src 'nonce-abc': Only scripts with matching nonce / 仅匹配nonce的脚本
- unsafe-inline / unsafe-eval: Weaken CSP; avoid in production / 削弱CSP；生产环境避免

2.9 CSRF, SSRF & SQL Injection / CSRF/SSRF/SQL注入 (Week 10)

CSRF (Cross-Site Request Forgery) (跨站请求伪造)
- Attack: Forges authenticated request from attacker's site / 从攻击者站点伪造认证请求
- Key: Browser automatically attaches target site's cookies / 浏览器自动附加目标站点的Cookie
- CSRF vs XSS: CSRF cannot READ response (SOP blocks); XSS can / CSRF不能读取响应；XSS可以
- XSS > CSRF: XSS can bypass CSRF tokens by reading them from page / XSS可通过从页面读取来绕过CSRF令牌

CSRF Defenses (CSRF防御)

Defense	Mechanism / 机制
CSRF Token	Server-issued secret token in forms / 服务器在表单中发放的秘密令牌
SameSite Cookie	Strict/Lax — cookie not sent on cross-site requests / Strict/Lax—跨站请求不发送Cookie
Referer/Origin check	Verify request originates from same site / 验证请求来源同站
Custom headers	X-Requested-With (AJAX only) / 仅AJAX

SSRF (Server-Side Request Forgery) (服务器端请求伪造)
- Attack: Server makes requests to attacker-specified URLs / 服务器向攻击者指定的URL发送请求
- Cloud metadata attack: Access http://169.254.169.254 to steal cloud credentials / 访问元数据端点窃取云凭证
- Defenses: Whitelist allowed destinations, block private IPs, disable URL redirects / 白名单允许目的地、阻止私有IP、禁用URL重定向

SQL Injection (SQL注入)
- Classic attack: ' OR 1=1 -- bypasses authentication / 绕过认证
- Types: In-band (UNION, error-based), blind (boolean, time-based) / 内联、盲注
- Impact: Data exfiltration, authentication bypass, RCE (via INTO OUTFILE) / 数据外泄、认证绕过、远程代码执行

SQL Injection Defenses (SQL注入防御)

Defense	Mechanism / 机制
Prepared Statements	Parameterized queries; code and data separated / 参数化查询；代码与数据分离
Input Validation	Whitelist allowed patterns / 白名单允许模式

Defense	Mechanism / 机制
Least Privilege	DB user has minimal permissions / 数据库用户最小权限
ORM	Frameworks escape by default / 框架默认转义

2.10 AI Security / AI安全 (Week 11)

AI Attack Categories (AI攻击类别)

Category	Description / 描述
Handcrafted Jailbreaks	Carefully crafted prompts to bypass alignment / 精心构造的提示词绕过对齐
Gradient-Based (GCG)	Automated adversarial suffix generation / 自动对抗后缀生成
Gradient-Free / Model-Based	Use one model to jailbreak another / 用一个模型越狱另一个
Direct Prompt Injection	Attacker controls prompt directly / 攻击者直接控制提示词
Indirect Prompt Injection	Attacker controls data that model reads (emails, docs) / 攻击者控制模型读取的数据
Side Channel Attacks	Extract info via timing, cache, power / 通过时序、缓存、功耗提取信息

Prompt Injection Hierarchy (提示注入层级)

- Direct: Attacker talks to model directly / 攻击者直接与模型对话
- Indirect: Attacker embeds instructions in external data (more dangerous) / 攻击者在外部数据中嵌入指令（更危险）

Defense in Depth for AI (AI的纵深防御)

Layer	Defense	Target
Input	Filtering, guard models	Malicious prompts
Model	Alignment, RLHF	Harmful generation
Output	Content filtering, fact-checking	Harmful responses
Tool	Least privilege, approval gates	Unauthorized actions
System	Sandboxing, logging, rate limits	Blast radius, forensics

Open Weight Security Risks (开源权重安全风险)

- Anyone can fine-tune to remove safety alignment / 任何人都可以微调移除安全对齐
- Deployment security matters more than model alignment / 部署安全比模型对齐更重要
- Key insight: Alignment is a thin veneer; prompt injection is the "new SQLi" / 对齐是薄层；提示注入是"新型SQL注入"

3. Key Formulas & Theorems / 重点公式与定理

AES Encryption (AES加密)

- Formula: C = AES_Enc(K, P); P = AES_Dec(K, C)
- Variables: K = key, P = plaintext, C = ciphertext / K=密钥, P=明文, C=密文
- Conditions: Block size = 128 bits; key = 128/192/256 bits / 块大小128位；密钥128/192/256位

CBC Mode Encryption (CBC模式加密)

- Formula: $C_i = Enc(K, P_i \oplus C_{i-1})$; $C_0 = IV$
- Variables: IV = initialization vector (must be random and unpredictable) / IV=初始化向量（必须随机且不可预测）
- Conditions: IV must never be reused with same key / IV不能与同一密钥重复使用

HMAC Construction (HMAC构造)

- Formula: $HMAC(K,m) = H((K \oplus opad) \| H((K \oplus ipad) \| m))$
- Variables: K = key, m = message, H = hash function, ipad/opad = inner/outer padding constants / K=密钥, m=消息, H=哈希函数
- Conditions: Key should be hash output length / 密钥应为哈希输出长度

Diffie-Hellman Key Exchange (DH密钥交换)

- Formula: $Shared secret = (g^b \bmod p)^a \bmod p = (g^a \bmod p)^b \bmod p = g^{ab} \bmod p$
- Variables: g = generator, p = prime, a/b = private values / g=生成元, p=素数, a/b=私有值
- Conditions: Vulnerable to MITM without authentication / 无认证时易受中间人攻击

Buffer Overflow Offset (缓冲区溢出偏移)

- Formula: offset = distance from buffer start to saved return address
- Variables: offset=buf_start+buffer_size+padding+saved_EBP_size / 偏移=缓冲区起始+缓冲区大小+填充+保存的EBP大小
- Conditions: Alignment may add padding; verify with GDB / 对齐可能添加填充；用GDB验证

Format String %n Write (格式化字符串%n写入)

- Formula: %n writes the number of characters printed so far into the pointed-to address
- Variables: %n = writes int count; %hn = writes short; %hhn = writes char / %n写入int计数；%hn写入short；%hhn写入char
- Conditions: Must have a pointer to target address on the stack / 栈上必须有指向目标地址的指针

4. Typical Questions & Solutions / 典型例题与解答

Question 1: ECB vs CBC (ECB与CBC模式对比)

Problem: ECB mode encrypts each block independently with the same key. Explain why this is insecure and how CBC mode solves the problem.

Solution: - ECB is insecure because identical plaintext blocks produce identical ciphertext blocks, revealing patterns in the data. An attacker can detect repetitions and even identify content from ciphertext patterns (e.g., encrypted images remain recognizable). - CBC solves this by XORing each plaintext block with the previous ciphertext block before encryption: $C_i = Enc(K, P_i \oplus C_{i-1})$. The first block uses a random IV. This ensures identical plaintext blocks produce different ciphertext blocks, eliminating pattern leakage.

- ECB不安全是因为相同的明文块产生相同的密文块，暴露数据模式。攻击者可以检测重复，甚至从密文模式识别内容。
- CBC通过对每个明文块在加密前与前一个密文块异或来解决：$C_i = Enc(K, P_i \oplus C_{i-1})$。第一个块使用随机IV。这确保相同明文块产生不同密文块。

Question 2: SSL Stripping & HSTS (SSL剥离与HSTS)

Problem: Explain how an SSL stripping attack works, and describe how HSTS defends against it. What limitation does HSTS have on a user's very first visit?

Solution: - SSL stripping (Moxie Marlinspike): MITM attacker intercepts the initial HTTP request and prevents the redirect to HTTPS. The victim continues on HTTP while the attacker proxies to the server via HTTPS. The victim sees no warning. - HSTS defends by telling the browser (via Strict-Transport-Security header) to always use HTTPS for that domain in future visits. - Limitation: On the very first visit, the browser hasn't received the HSTS header yet, so the first connection is still vulnerable to SSL stripping.

- SSL剥离：中间人攻击者拦截初始HTTP请求，阻止重定向到HTTPS。受害者继续使用HTTP，而攻击者通过HTTPS代理到服务器。受害者看不到警告。
- HSTS通过Strict-Transport-Security头告诉浏览器未来始终使用HTTPS。
- 限制：首次访问时浏览器尚未收到HSTS头，因此首次连接仍易受SSL剥离攻击。

Question 3: Set-UID Privilege Analysis (Set-UID权限分析)

Problem: Consider the root-owned Set-UID program:

```
int main(int argc, char *argv[]) {
    int fd = open("/etc/secret_notes", O_RDWR);   // Line A
    setuid(getuid());                              // Line B
    char cmd[256];
    sprintf(cmd, "cat %s", argv[1]);               // Line C
    system(cmd);                                   // Line D
    return 0;
}
```

User alice (UID 1000) runs: ./privnotes myfile.txt

(a) State the RUID and EUID at Line A, and again at Line B. Which line causes EUID to change? (b) After Line B, how can an attacker still exploit this program? (c) Give two concrete fixes.

Solution: (a) At Line A: RUID=1000 (alice), EUID=0 (root). After Line B (setuid(getuid())): RUID=1000, EUID=1000. Line B causes EUID to change because setuid(getuid()) drops root privileges by setting EUID to the real UID.

(b) Two vulnerabilities remain: 1. File descriptor leak (Line A→D): fd to /etc/secret_notes is still open after privilege drop. Attacker can pass /proc/self/fd/3 as argv[1] to read the protected file via the still-open descriptor. 2. Command injection (Line C→D): system() executes user-supplied argv[1] through the shell. Attacker can pass myfile.txt; cat /etc/secret_notes or use shell metacharacters.

(c) Fixes: 1. For FD leak: Close the file descriptor before calling system(), or set FD_CLOEXEC flag so it's automatically closed on exec. 2. For command injection: Replace system() with execvp() which does not invoke a shell, or use execve() with explicit argument arrays.

- (a) Line A处：RUID=1000, EUID=0 (root)。Line B后：RUID=1000, EUID=1000。Line B导致EUID改变。
- (b) 两个漏洞：文件描述符泄漏（特权降级后fd仍开放）和命令注入（system()使用用户输入）。
- (c) 修复：关闭fd或设置FD_CLOEXEC；用execvp()替代system()。

Question 4: Kaminsky DNS Attack (Kaminsky DNS攻击)

Problem: In the Kaminsky DNS cache poisoning attack, the attacker queries for random subdomains (e.g., xyz123.example.com) instead of the target hostname directly. Why is this the key insight?

Solution: C: Random subdomains are never cached, so the attacker gets unlimited fresh attempts without waiting for TTL expiry. If the attacker queried the target hostname directly, the legitimate response would be cached and the attacker would have to wait for TTL to expire before trying again. Random subdomains ensure the resolver always has to forward the query to the authoritative server, giving the attacker unlimited chances to race with the legitimate response.

- 随机子域名永远不会被缓存，因此攻击者获得无限新鲜尝试而无需等待TTL过期。如果直接查询目标主机名，合法响应会被缓存，攻击者必须等待TTL过期才能再试。随机子域名确保解析器总是必须将查询转发到权威服务器。

Question 5: DNS Spoofing Required Fields (DNS欺骗必要字段)

Problem: Which combination of fields must a remote attacker correctly guess to successfully spoof a DNS response?

Solution: B: 16-bit Transaction ID + 16-bit source port. The attacker must match both the Transaction ID (to identify the correct query) and the source port (to direct the response to the correct resolver port). Older resolvers used fixed source port 53, making only Transaction ID guessing necessary (65,536 possibilities). Modern resolvers randomize source ports, increasing the search space to 2^{32}.

- 16位事务ID + 16位源端口。攻击者必须同时匹配事务ID和源端口。旧解析器使用固定源端口53，使事务ID猜测成为唯一障碍（65,536种可能）。现代解析器随机化源端口，将搜索空间增加到2^{32}。

Question 6: Stateful vs Stateless Firewall (有状态vs无状态防火墙)

Problem: Explain why a stateless packet filter that allows outbound TCP/80 and inbound TCP from source port 80 is vulnerable, and how a stateful firewall prevents this.

Solution: A stateless firewall allows any inbound packet with source port 80, including unsolicited packets from an attacker who sets their source port to 80. An attacker can initiate connections to internal services by sending packets FROM port 80, and the stateless filter would allow them through.

A stateful firewall maintains a connection tracking (conntrack) table. It only allows inbound packets that match an existing outbound connection in the ESTABLISHED state. Since no outbound connection was initiated to the attacker's address, the unsolicited inbound packet is dropped.

- 无状态防火墙允许任何源端口80的入站包，但若攻击者从端口80发起的未请求的包。有状态防火墙维护连接跟踪表，仅允许匹配已有出站连接的入站包。

Question 7: VPN Packet Flow (VPN包流程)

Problem: A browser on a VPN client sends a packet to 10.0.8.5. Describe the packet's journey through the VPN.

Solution: 1. The browser sends a packet to 10.0.8.5 (private IP). / 浏览器发送包到10.0.8.5 2. The kernel's routing table directs 10.0.8.x traffic to the tun0 interface. / 内核路由表将流量导向tun0 3. The VPN application reads the raw IP packet from tun0, encrypts it with AES-256. / VPN应用从tun0读取原始IP包，用AES-256加密 4. Wraps it in a new UDP packet: outer src=client's public IP, dst=VPN server's public IP. / 封装在新的UDP包中 5. Sends the encrypted tunnel packet out through eth0. / 通过eth0发出加密隧道包

Question 8: DNS Exfiltration via DoH (通过DoH的DNS数据外泄)

Problem: A compromised host uses DNS-over-HTTPS (DoH) for data exfiltration. (a) Why can't a firewall detect this? (b) What's a better approach than blocking all outbound UDP/53? (c) What detection strategies exist?

Solution: (a) DoH encapsulates DNS queries inside HTTPS on port 443, making them indistinguishable from normal web browsing traffic. The firewall cannot differentiate between legitimate HTTPS and DoH exfiltration.

(b) Instead of blocking all outbound DNS (which breaks all services), force DNS through a designated internal resolver: whitelist only the resolver's IP for outbound port 53, and deploy DNS-layer monitoring (entropy analysis, query-rate limits, DNS firewall like RPZ).

(c) Detection strategies: monitor connections to known public DoH resolver IPs (1.1.1.1, 8.8.8.8 on port 443); deploy enterprise TLS inspection proxy; use endpoint detection (EDR) to flag processes making DoH requests.

- (a) DoH将DNS查询封装在HTTPS的443端口中，与正常Web流量无法区分。
- (b) 强制DNS通过指定内部解析器，白名单仅允许解析器IP的出站53端口，部署DNS层监控。
- (c) 监控已知DoH解析器IP的连接、部署TLS检测代理、使用EDR终端检测。

Question 9: TUN vs TAP Mode (TUN与TAP模式对比)

Problem: In VPN site-to-site configuration, when would you prefer TAP (Layer 2) over TUN (Layer 3)?

Solution: TUN (Layer 3, IP packets only) is preferred for most site-to-site routing because it's more efficient (no Ethernet overhead, no MAC addresses, no ARP). TAP (Layer 2, Ethernet frames) is preferred when the two sites need to appear as one broadcast domain --- e.g., when applications rely on Layer 2 discovery protocols, ARP resolution across sites, or DHCP that must span both locations.

- TUN（三层，仅IP包）更高效，用于大多数站点间路由。TAP（二层，以太网帧）在两个站点需要同一广播域时使用，如应用依赖二层发现协议、跨站点ARP或DHCP。

Question 10: XSS vs CSRF (XSS与CSRF对比)

Problem: Compare XSS and CSRF: which is more powerful and why?

Solution: XSS is more powerful because: - XSS code runs in the victim's origin → can both SEND requests AND READ responses - CSRF can only SEND requests; cannot READ responses (blocked by SOP) - XSS can bypass CSRF tokens by reading them from the page DOM - XSS can steal cookies (if not HttpOnly), perform actions as the victim, and propagate as a worm

Aspect	CSRF	XSS
Can send requests?	Yes	Yes
Can read responses?	No (SOP blocks)	Yes (same origin)
Can bypass CSRF tokens?	No	Yes (can read tokens)

- XSS更强大：XSS代码在受害者源中运行→可以发送请求并读取响应。CSRF只能发送请求，不能读取响应（被SOP阻止）。XSS可以通过从页面DOM读取来绕过CSRF令牌。

Question 11: SQL Injection Prevention (SQL注入防御)

Problem: Why are prepared statements effective against SQL injection? Give an example.

Solution: Prepared statements separate SQL code from data. The query structure is fixed before any user input is inserted, and parameters are treated as literal values, not executable SQL.

Example --- Vulnerable:

```
query = "SELECT * FROM users WHERE name = '" + user_input + "'"
# Input: ' OR 1=1 -- → SELECT * FROM users WHERE name = '' OR 1=1 --'
```

Example --- Safe:

```
query = "SELECT * FROM users WHERE name = ?"
cursor.execute(query, (user_input,))  # Parameter treated as data, not SQL
```

Even if user_input = "' OR 1=1 --", the database treats it as a literal string to match against the name column, not as SQL code.

- 预处理语句将SQL代码与数据分离。查询结构在任何用户输入插入之前已固定，参数被视为字面值而非可执行SQL。

Question 12: AI Prompt Injection (AI提示注入)

Problem: Explain the difference between direct and indirect prompt injection. Which is more dangerous and why?

Solution: - Direct prompt injection: Attacker directly controls the prompt sent to the model (e.g., typing "ignore previous instructions" in a chat interface). - Indirect prompt injection: Attacker embeds malicious instructions in external data that the model reads (e.g., hidden text in an email, webpage, or document that the AI agent processes).

Indirect prompt injection is more dangerous because: 1. The user doesn't see the malicious instructions (they're embedded in data) 2. The model may trust the data source implicitly 3. It scales --- one poisoned document can affect every user whose AI reads it 4. Harder to detect and prevent

- 直接注入：攻击者直接控制发给模型的提示词。间接注入：攻击者在模型读取的外部数据中嵌入恶意指令。间接注入更危险，因为用户看不到恶意指令，模型可能信任数据源，且一个有毒文档可影响所有用户。

5. Exam Tips / 考试要点

Key Comparison Tables to Remember (需记住的关键对比表)

1. ECB vs CBC vs CTR vs GCM
2. Symmetric vs Asymmetric encryption
3. Reflected vs Stored vs DOM-based XSS
4. CSRF vs XSS
5. TUN vs TAP
6. Stateful vs Stateless firewall
7. DNSSEC vs DoH vs DoT
8. Ret2Libc vs ROP
9. DAC vs MAC vs RBAC

Common Pitfalls (常见陷阱)

- ASLR doesn't prevent overflow --- it just makes guessing addresses harder / ASLR不阻止溢出---只是让猜测地址更难
- NX prevents shellcode on stack, but doesn't prevent code reuse attacks / NX阻止栈上shellcode，但不阻止代码重用攻击
- CSRF tokens are useless against XSS / CSRF令牌对XSS无效
- HttpOnly prevents JS cookie access but doesn't stop authenticated requests / HttpOnly阻止JS访问Cookie但不阻止认证请求
- DNSSEC doesn't encrypt DNS data / DNSSEC不加密DNS数据

Week 12: AI for Cybersecurity / AI赋能网络安全

12.1 The Phase Transition (相变)

- Definition (定义)：AI in cybersecurity is a qualitative shift, not incremental. Both attackers and defenders gain speed, scale, and adaptability simultaneously. / AI在网络安全中是质变而非渐变，攻防双方同时获得速度、规模和适应性优势。
- Key Comparison (关键对比)：

Property / 属性	Before AI / AI之前	After AI / AI之后
Speed / 速度	Human reaction (minutes- hours) / 人类反应	Machine reaction (milliseconds) / 机器反应
Scale / 规模	One analyst per investigation / 一位分析师一项调查	One agent per alert, thousands in parallel / 一个Agent一个告警，数千并行
Adaptability / 适应性	Rules updated quarterly / 规则季度更新	Models retrained continuously / 模型持续重训
Entry barrier / 准入门槛	Years of training / 多年训练	Natural-language instruction / 自然语言指令

12.2 Intent-Based Security Operations (基于意图的安全运营)

- Definition (定义)：AI bridges the gap between intent and execution — describe goals in natural language, agent selects/configures tools and orchestrates multi-step workflows. / AI弥合意图与执行之间的鸿沟。
- Key Points (要点)：Works for both offense ("find a way into this network") and defense ("detect lateral movement in these logs"). The barrier to entry has collapsed for both sides. / 攻防双方均可用；准入门槛已崩塌。

12.3 The Fundamental Tension (根本矛盾)

- Definition (定义)：AI agents are probabilistic; cybersecurity is consequential. / AI Agent是概率性的；网络安全是后果严重的。
- Key Issues (关键问题)：
- Hallucination (幻觉)：Agent reports non-existent vulnerabilities or misses real ones / 报告不存在的漏洞或遗漏真实漏洞
- Non-determinism (非确定性)：Same input may produce different actions on different runs / 相同输入不同运行产生不同动作
- Instruction/data confusion (指令/数据混淆)：Agent can't reliably distinguish commands from content (prompt injection) / 无法可靠区分指令与内容
- No formal guarantees (无形式化保证)：Cannot prove agent won't cause unintended effects / 无法证明Agent不会导致意外效果

12.4 AI-Powered Offense (AI赋能攻击)

- Reconnaissance (侦察)：AI-driven OSINT gathering, automated port scanning, vulnerability correlation / AI驱动的开源情报收集、自动端口扫描、漏洞关联
- Weaponization (武器化)：Automated exploit generation, polymorphic malware / 自动化利用生成、多态恶意软件
- Social Engineering (社会工程)：Deepfake voice/video, personalized phishing at scale / 深伪语音/视频、大规模个性化钓鱼
- Lateral Movement (横向移动)：AI-guided privilege escalation and path finding / AI引导的提权和路径发现
- Key Points (要点)：AI lowers the skill barrier for sophisticated attacks; enables fully autonomous attack chains. / AI降低了高级攻击的技能门槛。

12.5 AI-Powered Defense (AI赋能防御)

- Threat Detection (威胁检测)：ML-based anomaly detection, behavioral analytics / 基于ML的异常检测、行为分析
- Automated Response (自动响应)：SOAR (Security Orchestration, Automation, Response) / 安全编排、自动化与响应
- Vulnerability Management (漏洞管理)：AI-assisted code review, priority scoring / AI辅助代码审查、优先级评分
- Threat Intelligence (威胁情报)：NLP on threat reports, IOC extraction / 威胁报告NLP处理、IOC提取
- Key Points (要点)：AI matches attacker speed; enables 24/7 monitoring at scale impossible for humans. / AI匹配攻击者速度；实现人类不可能达到的全天候大规模监控。

12.6 Agent Reliability Problem (Agent可靠性问题)

- Definition (定义)：Making AI tools trustworthy enough to deploy in high-stakes security environments. / 使AI工具足够可信以部署在高风险安全环境中。
- Key Aspects (关键方面)：
- Human-in-the-loop vs fully autonomous / 人在环中 vs 完全自主
- Confidence calibration and uncertainty quantification / 置信度校准与不确定性量化
- Audit trails and explainability / 审计追踪与可解释性
- Sandboxing and containment / 沙箱与隔离

12.7 Red, Blue, and Purple Teaming (红蓝紫队演练)

- Red Team (红队)：AI-assisted attack simulation, automated penetration testing / AI辅助攻击模拟、自动化渗透测试
- Blue Team (蓝队)：AI-enhanced detection and response / AI增强检测与响应
- Purple Team (紫队)：Continuous adversarial improvement — red and blue collaborate to strengthen defenses iteratively / 持续对抗性改进——红蓝协作迭代加强防御
- Key Points (要点)：AI enables continuous, automated adversarial testing rather than periodic manual exercises. / AI使对抗测试从定期人工变为持续自动化。

12.8 Limits and the Human Role (局限性与人的角色)

- Where AI Fails (AI失败之处)：Novel zero-day attacks with no training data; ethical/legal judgment; understanding attacker intent; strategic decision making / 无训练数据的新型零日攻击；伦理/法律判断；理解攻击者意图；战略决策
- Human Role (人的角色)：Oversight, ethical decisions, policy creation, handling edge cases, validating AI outputs / 监督、伦理决策、策略制定、处理边缘情况、验证AI输出
- Key Points (要点)：AI amplifies human capabilities but cannot replace human judgment in consequential decisions. / AI放大人类能力但不能替代后果严重决策中的人类判断。

Typical Questions for Week 12 / Week 12典型例题

Question: AI Offense vs Defense Asymmetry (AI攻防不对称性)

Problem (问题)：Explain why the introduction of AI into cybersecurity creates an asymmetric advantage and which side benefits more initially.

Solution (解答)：- Offense initially benefits more because: (1) attackers only need to find ONE vulnerability, defenders must protect ALL surfaces; (2) AI-generated attacks can be tested privately before deployment; (3) defenders must avoid false positives while attackers don't care about precision. / 攻击方初期获益更多，因为攻击者只需找一个漏洞，防御者须保护所有面。 - However, over time defense catches up because: (1) defenders have more data (logs, traffic); (2) defenders can retrain continuously on new attack patterns; (3) automated response can match attack speed. / 但长期防御方追赶上来，因为防御者有更多数据且可持续重训。

Question: Agent Reliability (Agent可靠性)

Problem (问题)：Why is deploying AI agents in cybersecurity particularly challenging compared to other AI applications?

Solution (解答)：1. Consequences are severe: A hallucinating agent might block legitimate traffic or miss real attacks / 后果严重：幻觉Agent可能阻断合法流量或遗漏真实攻击 2. Adversarial environment: Attackers actively try to fool AI (adversarial examples, prompt injection) / 对抗环境：攻击者主动欺骗AI 3. Non-determinism is unacceptable: Security operations require predictable, auditable behavior / 安全运营需要可预测、可审计的行为 4. Data/instruction confusion: In security contexts, processing attacker-controlled input that may contain injected instructions / 处理攻击者控制的输入可能包含注入的指令